

Lab 05 — Questions

Question 1 [Analysis]

A team ships its Spring Boot API as a 950 MB image built from `maven:3.9-eclipse-temurin-21`. The security officer blocks the production release. Give **three** arguments that have nothing to do with disk space, then say which one is hardest to fix without a multi-stage build.

Question 2 [Understanding]

A multi-stage Dockerfile has three `FROM`s. Which one produces the final image, and what happens to the others? If no `COPY --from` references the second stage, what does Buildah do — and how does that show in the output of `podman build`?

Question 3 [Diagnosis]

A developer writes:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
COPY . /app
WORKDIR /app
RUN mvn package -DskipTests

FROM docker.io/library/eclipse-temurin:21-jre-alpine
COPY --from=build /app /app
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

The final image weighs 420 MB instead of the expected 210 MB, and the source code is still inside. Explain the mistake and fix it in one line.

Question 4 [Analysis]

To "save time", your colleague containerises `ng serve`: the image holds Node and the project sources, and starts the Angular development server on port 4200. It works fine in staging. Give four reasons to reject this image for production, then describe in two sentences what to do instead.

Question 5 [Analysis]

A team moves its image from `eclipse-temurin:21-jre` (Ubuntu) to `eclipse-temurin:21-jre-alpine` and saves 120 MB. Two weeks later, a PDF generation job fails in production with an `UnsatisfiedLinkError`. Explain the likely cause, name the command that would have revealed the difference in each image, and describe how this migration should have been run.

Question 6 [Understanding]

Why is multi-stage called the only **reliable** protection against build secrets, when you could also delete the file with `rm`? In which case does multi-stage **not** protect you?

Question 7 [Analysis]

Compare these two strategies for a 50 MB Spring Boot JAR, looking at the **deployment time** of a one-line fix: (a) `COPY target/api.jar app.jar`, (b) layered extraction (`-Djarmode=tools ... extract`

`--layers`). Estimate roughly how much data moves in each case, and explain why (a) is still good enough for many companies.

Question 8 [Diagnosis]

A CI build jumps from 90 seconds to 7 minutes after moving to a new agent, even though no file changed. The Dockerfile is correctly ordered (dependencies before code). Explain why, and give two mechanisms that recover the 90 seconds — stating, for an agent that builds with rootless Podman, where the cache lives.

Question 9 [Analysis]

A *distroless* image sharply reduces the attack surface. Name two operational capabilities you concretely lose, and say how a team usually compensates for each.

Question 10 [Understanding]

`RUN --mount=type=cache,target=/root/.m2 mvn package`: where is that data stored, and why does it never show up in the final image? How does it differ from a `VOLUME`? And what happens if you leave out the `# syntax=docker/dockerfile:1` line — first under Docker, then under Podman?

Question 11 [Analysis]

A developer claims: "Multi-stage is pointless for Angular — the result is just static files anyway." Answer them by describing what the image would contain without multi-stage, and how much size is at stake.

Question 12 [Diagnosis]

The following Dockerfile fails with `COPY failed: ... /app/dist: no such file or directory`:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /src
COPY . .
RUN npm ci && npm run build

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Find the error, then name the `podman build` command that would let you inspect the actual content of the `build` stage instead of guessing.

Question 13 [Analysis]

Your company requires that failing unit tests break the image build. Where do you place `RUN mvn test` in a multi-stage Dockerfile, and what is the drawback compared with running the tests upstream in CI?

Question 14 [Analysis]

Two images of the same application: one is 250 MB in a single layer, the other 280 MB in five layers, four of them stable. Which one deploys faster when the application code is updated? Justify your answer, and say in which situation it reverses.